National Institute of Technology Patna
Department of Electronics and Communication Engineering

Minor Project — Architecture & Module Specification Report

Defensive KYC:

Robust Detection of Synthetic Camera Streams and Replay Attacks

Submitted By

Manmohan Vishwakarma
Roll No: 2304172 — B.Tech, ECE

Ankur Raj
Roll No: 2304132 — B.Tech, ECE

Supervised by:

Dr.Manpuran Mahto
Department of ECE, NIT Patna

March 22, 2026 — Version 1.0

Contents

1 Project Philosophy: From Deepfake Detection to Presentation Attack Defence **2**
1.1 Why Standard Deepfake Detectors Are Insufficient 2
1.2 Our Engineering Philosophy: The Layered PAD Gateway 2

2 System Flow: The Waterfall Execution Pipeline **3**
2.1 Latency Budget 5

3 Module Breakdown **5**
3.1 Stage 0: Stream Telemetry Pre-Check 5
3.2 Stage 1: PRNU Sensor Forensics 6
3.2.1 Physical Principle 6
3.2.2 Detection Procedure 6
3.3 Stage 2: Moiré / Replay Attack Detector 7
3.3.1 Physical Principle 7
3.3.2 Detection Procedure 8
3.4 Stage 3: rPPG Biological Liveness Gate 8
3.4.1 Physiological Basis 8
3.4.2 Signal Extraction: CHROM Algorithm 9
3.5 Stage 4: Frequency-Temporal Cross-Attention (FTCA) Engine 10
3.5.1 Motivation 10
3.5.2 Architecture 10
3.5.3 Training Objective 12

4 Score Fusion and Final Decision **12**

5 In-the-Wild Evaluation Dataset: NIT-KYC Wild (NKW-400) **13**
5.1 Motivation and Contribution 13
5.2 Dataset Construction Protocol 13
5.3 Significance 14

6 Codebase Architecture and Rationale **14**
6.1 Directory Structure 15
6.2 Design Rationale 15

7 Evaluation Plan **16**
7.1 Metrics 16
7.2 Ablation Study 16

8 Summary of Novel Contributions **17**

1 Project Philosophy: From Deepfake Detection to Presentation Attack Defence

1.1 Why Standard Deepfake Detectors Are Insufficient

A common misconception is that KYC security reduces to a single question: *"Is this face real or generated?"* In practice, the threat landscape is significantly broader. Adversaries targeting banking and fintech KYC systems employ what the ISO/IEC 30107-3 standard formally classifies as **Presentation Attacks (PA)** — any attempt to subvert a biometric capture subsystem using a fraudulent artefact.

Table 1: Presentation Attack Taxonomy vs. System Response

Attack Class	Mechanism	*Defeated By*
Screen Replay	Prerecorded video played on iPad/monitor to a physical camera	*Moiré / FFT detector*
Virtual Camera Injection	OBS, ManyCam inject synthetic stream directly into OS camera driver	*PRNU + Telemetry*
Real-time Deepfake	Live GPU face-swap (DeepFaceLive, Roop)	*FTCA engine + rPPG*
Fully Synthetic AI Video	Diffusion/GAN talking-head generation	*FTCA engine + rPPG*

1.2 Our Engineering Philosophy: The Layered PAD Gateway

Rather than relying on one expensive neural network to catch everything, we engineer a **cascade of increasingly expensive checks**. The key insight is:

Core Design Principle

"Check the physical, biological, and mathematical authenticity of the video stream ***before*** *it reaches the deep learning layer."*

Each layer is designed to be the cheapest possible check that can decisively reject a specific class of attack. A fraudulent stream is rejected at the first layer that catches it, saving GPU compute and minimising latency.

This is called a **Waterfall Execution Strategy**: early stages act as fast, deterministic mathematical filters. The deep learning engine (FTCA) only runs on streams that pass all prior checks — i.e., streams that are already confirmed to originate from a physical camera carrying a live human face.

2 System Flow: The Waterfall Execution Pipeline

Figure 1 illustrates the complete execution pipeline. Read it top-to-bottom: each stage either **rejects the stream immediately** (dashed red arrow) or **passes it to the next stage** (solid blue arrow).

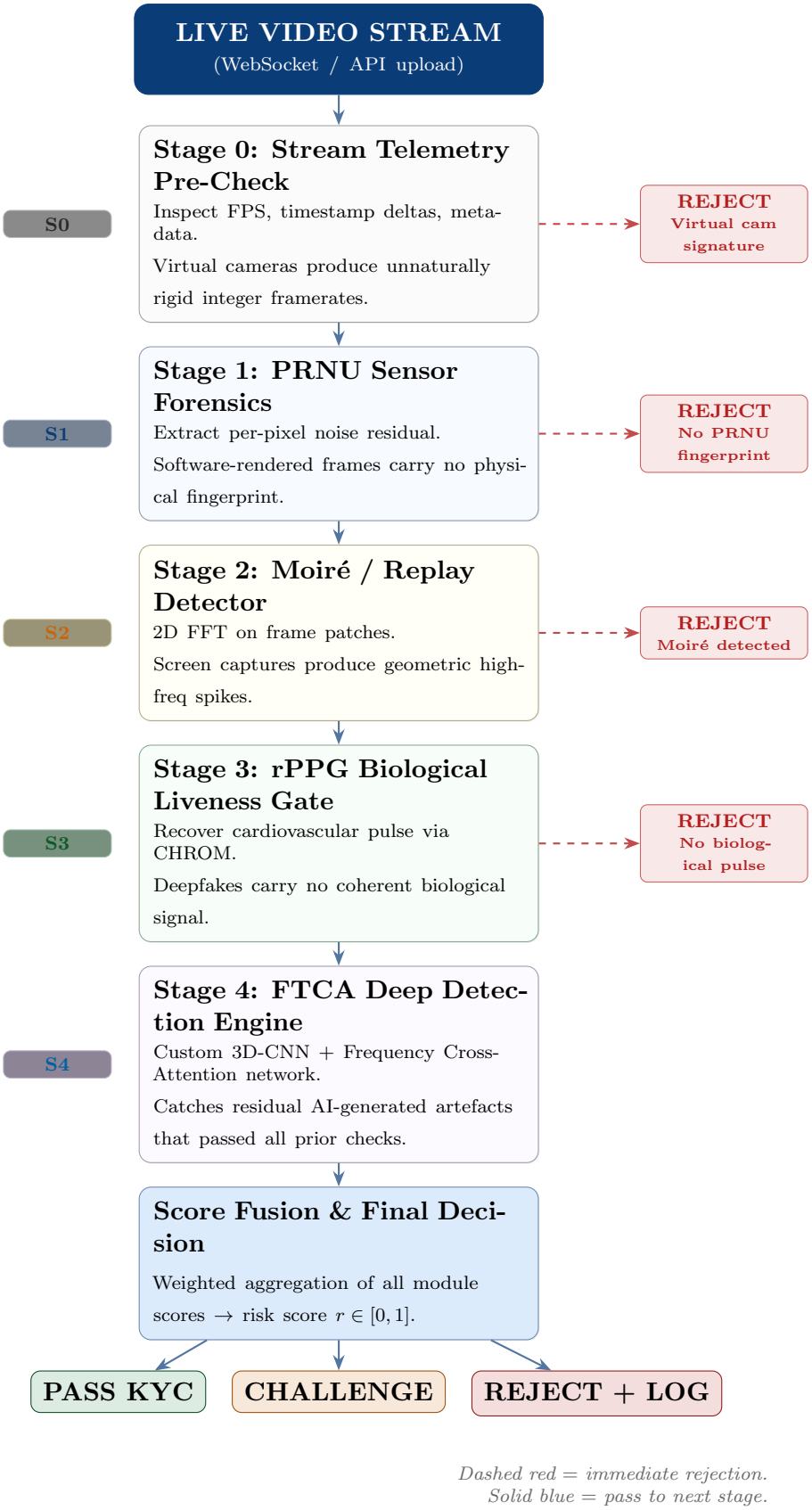

Figure 1: Waterfall Execution Pipeline — Streams are rejected at the earliest and cheapest possible stage.

2.1 Latency Budget

The waterfall design provides a strict latency budget. Most fraudulent streams are caught by cheap CPU-based stages (S0–S2) and never reach the GPU.

Table 2: Per-Stage Latency Budget (16-frame window, 1080p input)

Stage	Module	Hardware	Target Latency	Cumulative
S0	Stream Telemetry	CPU	< 5 ms	5 ms
S1	PRNU Forensics	CPU	< 25 ms	30 ms
S2	Moiré / Replay	CPU	< 20 ms	50 ms
S3	rPPG Liveness	CPU	< 20 ms	70 ms
S4	FTCA Engine	GPU	< 70 ms	140 ms
—	Fusion + Output	CPU	< 10 ms	**150 ms**

3 Module Breakdown

3.1 Stage 0: Stream Telemetry Pre-Check

Stage 0 — Stream Telemetry

Cost: $\mathcal{O}(1)$ — pure metadata parsing, no frame processing.
Catches: Lazily configured virtual cameras (rigid FPS, missing EXIF, unnatural timestamp cadence).

Virtual camera software such as OBS outputs a rigidly synthetic stream: timestamps are integer-spaced, FPS is exactly `30.000` or `60.000` with zero jitter, and the frame metadata carries no camera EXIF fields (ISO, exposure time, lens model). Physical cameras produce natural micro-jitter in inter-frame timestamps arising from sensor readout variation and clock drift.

Detection signal: Compute the coefficient of variation of inter-frame timestamp deltas $\{\Delta t_i\}_{i=1}^{N}$:

$$\mathrm{CV} = \frac{\sigma(\Delta t)}{\mu(\Delta t)} \tag{1}$$

Virtual camera streams consistently yield $\mathrm{CV} < \epsilon_{\text{tele}}$ (empirically, $\approx 10^{-4}$ vs. 10^{-2}–10^{-1} for physical cameras).

Pseudocode

```
def telemetry_check(frame_timestamps: list[float]) -> bool:
    deltas = np.diff(frame_timestamps)
    cv = deltas.std() / deltas.mean()
    fps_declared = 1.0 / deltas.mean()
    if cv < TELEMETRY_CV_THRESHOLD:
        return REJECT  # unnaturally rigid timing
    if fps_declared in {24.0, 30.0, 60.0} and cv < 0.001:
        return REJECT  # suspiciously perfect integer FPS
    return PASS
```

3.2 Stage 1: PRNU Sensor Forensics

Stage 1 — PRNU Sensor Forensics

Cost: Medium — BM3D/DnCNN denoising on N frames.
Catches: Any stream rendered in software (virtual cameras, fully synthetic AI video).

3.2.1 Physical Principle

Manufacturing imperfections in CMOS/CCD sensor arrays produce a deterministic, per-pixel gain variation called **Photo-Response Non-Uniformity (PRNU)**. Every physical camera has a unique PRNU fingerprint $\mathbf{K}$ that persists across all frames it captures, modelled as:

$$I_c(x, y) = I_c^0(x, y)\big(1 + \mathbf{K}(x, y)\big) + N(x, y) \tag{2}$$

where I_c^0 is the true scene signal, $\mathbf{K}$ is the PRNU map, and N is zero-mean random noise. Frames generated by software renderers have *no such fingerprint*: they contain only I_c^0 with synthetic noise that does not match any physical sensor.

3.2.2 Detection Procedure

Step 1 — Noise residual extraction. Apply a denoising filter $F(\cdot)$ to suppress scene content:

$$\hat{W}_i = I_i - F(I_i) \tag{3}$$

Step 2 — Fingerprint estimation. Average residuals over n frames to suppress random noise:

$$\hat{\mathbf{K}} = \frac{1}{n} \sum_{i=1}^{n} \hat{W}_i \tag{4}$$

Step 3 — Authenticity test. Compute the energy of the estimated fingerprint:

$$E_{\text{PRNU}} = \|\hat{\mathbf{K}}\|_F \tag{5}$$

A real camera stream yields $E_{\text{PRNU}} > \tau_{\text{PRNU}}$. Software-rendered streams yield $E_{\text{PRNU}} \approx 0$. For cross-session verification (optional), compute Pearson correlation against a pre-enrolled reference $\mathbf{K}_{\text{ref}}$:

$$\rho = \frac{\hat{\mathbf{K}} \cdot \mathbf{K}_{\text{ref}}}{\|\hat{\mathbf{K}}\| \, \|\mathbf{K}_{\text{ref}}\|} \tag{6}$$

□

FIGURE PLACEHOLDER

Side-by-side comparison of PRNU noise residual maps:
Left: Physical camera (structured fingerprint visible).
Right: OBS virtual camera (flat / structureless residual).

Suggested: matplotlib heatmap of $\hat{\mathrm{K}}$ for both cases.

Figure 2: PRNU noise residuals: physical camera (left) vs. software-rendered frame from OBS (right). Real cameras show a consistent structured fingerprint.

3.3 Stage 2: Moiré / Replay Attack Detector

Stage 2 — Replay / Moiré Detector

Cost: Low — 2D FFT on grayscale frame patches.
Catches: Screen replay attacks (phone/iPad/monitor presented to camera).

3.3.1 Physical Principle

When a physical camera captures another digital screen, the camera's pixel sampling grid aliases against the screen's pixel grid, producing **Moiré patterns** — periodic interference fringes visible as diagonal bands. In the frequency domain, these appear as anomalous geometric spikes at spatial frequencies determined by the aliasing relationship.

3.3.2 Detection Procedure

For each frame I_t, compute the 2D DFT magnitude spectrum:

$$M_t(u,v) = \left| \sum_{x,y} I_t^{\text{gray}}(x,y)\, e^{-2\pi j(ux/W + vy/H)} \right| \tag{7}$$

Apply log-compression $\tilde{M}_t = \log(1 + M_t)$ and suppress the DC component. Compute the **radial spectral energy** at mid-high spatial frequencies $[f_{\min}, f_{\max}]$ and test for sharp localised peaks using a threshold on the peak-to-mean ratio:

$$\text{PMR} = \frac{\max_{(u,v)\in\mathcal{R}} \tilde{M}_t(u,v)}{\text{mean}_{(u,v)\in\mathcal{R}} \tilde{M}_t(u,v)} \tag{8}$$

A PMR exceeding τ_{PMR} indicates a screen-recording artefact.

□

FIGURE PLACEHOLDER

2D FFT magnitude spectrum comparison:
Left: Clean live camera feed (diffuse spectral energy, no spikes).
Right: Screen-recorded video (geometric spike pattern from Moiré).

Generate with: `np.fft.fft2` *on a real vs. replayed frame, plot with* `plt.imshow(np.log1p(np.abs(np.fft.fftshift(fft2))))`

Figure 3: 2D FFT magnitude spectra. Clean feed (left): diffuse, isotropic energy. Screen-recorded feed (right): sharp geometric spikes from Moiré aliasing pattern.

3.4 Stage 3: rPPG Biological Liveness Gate

Stage 3 — rPPG Liveness Gate

Cost: Low–medium — signal processing on ∼300 frames (≈10 s).
Catches: Deepfake videos, AR face filters, and any synthetic stream with no biological pulse.
Key property: Patent anchor. No existing KYC product deploys this check.

3.4.1 Physiological Basis

Blood volume pulsation (BVP) from the cardiac cycle causes microscopic, *imperceptible to the human eye* colour fluctuations in exposed facial skin. These arise because oxyhaemoglobin and deoxyhaemoglobin have different optical absorption spectra in the

visible range, modulated by each heartbeat. Remote photoplethysmography (rPPG) recovers this signal from a standard RGB video.

Why deepfakes fail: Current generative models (GANs, Diffusion models, face-swap networks) optimise for perceptual realism against human vision. They have *no training signal* to reproduce a coherent cardiovascular pulse. Even if a generator accidentally produces low-frequency intensity variation, it will not be confined to the physiologically valid cardiac band, nor will it exhibit the cross-channel chrominance consistency of a real BVP signal.

3.4.2 Signal Extraction: CHROM Algorithm

Step 1 — ROI tracking. MediaPipe Face Mesh tracks 468 landmarks per frame. Cheek and forehead patches are selected as the rPPG ROI (minimal hair/shadow coverage).

Step 2 — Spatially averaged chrominance signals. For each frame t, compute the mean RGB value over the ROI: $\bar{\mathbf{C}}(t) = [\bar{R}(t),\, \bar{G}(t),\, \bar{B}(t)]^\top$.

Step 3 — CHROM projection.

$$X(t) = \bar{R}(t) - \bar{G}(t) \tag{9}$$

$$Y(t) = \tfrac{1}{2}\bar{R}(t) + \tfrac{1}{2}\bar{G}(t) - \bar{B}(t) \tag{10}$$

$$S_{\text{CHROM}}(t) = X(t) - \frac{\sigma(X)}{\sigma(Y)}\, Y(t) \tag{11}$$

Step 4 — Bandpass filtering. Apply a zero-phase Butterworth bandpass filter over $[0.7,\, 4.0]$ Hz (corresponding to 42–240 bpm), yielding the pulse waveform $p(t)$.

Step 5 — SNR-based liveness gate. Compute the DFT $P(f)$ of $p(t)$. Define the valid BVP band $\mathcal{B} = [0.7, 4.0]$ Hz. The pulse SNR is:

$$\text{SNR}_{\text{pulse}} = 10 \log_{10} \left(\frac{\sum_{f \in \mathcal{B}} |P(f)|^2}{\sum_{f \notin \mathcal{B}} |P(f)|^2} \right) \tag{12}$$

If $\text{SNR}_{\text{pulse}} < \tau_{\text{rPPG}}$, the session is **hard-rejected**.

FIGURE PLACEHOLDER

Two-panel plot comparing rPPG signals:

Panel A (top): Time-domain pulse waveform $p(t)$
— Real human: clear periodic oscillation at $\sim$1.1 Hz (66 bpm)
— Deepfake video: flat / aperiodic noise signal

Panel B (bottom): Power Spectral Density (Welch's method)
— Real human: sharp peak in $[0.7, 4.0]$ Hz band, high SNR
— Deepfake: no dominant peak, low SNR across all frequencies

Generate using UBFC-rPPG dataset for real, FaceForensics++ for fake.

Figure 4: rPPG pulse signal comparison. Top: time-domain waveform. Bottom: power spectral density. Real human (blue) shows a dominant cardiac-frequency peak; deepfake (red) shows no coherent pulse.

3.5 Stage 4: Frequency-Temporal Cross-Attention (FTCA) Engine

Stage 4 — FTCA Deep Detection Engine

Cost: High — GPU inference on 16-frame clip.
Catches: Sophisticated deepfakes that survived all prior checks.
Key property: Novel architecture. Custom-designed — the core research contribution.

3.5.1 Motivation

Streams that reach Stage 4 have already passed: temporal telemetry, physical sensor verification, replay artefact screening, and biological liveness gating. These are the hardest adversarial samples. Standard frame-based classifiers fail here because they cannot correlate *spatial pixel inconsistencies* with *frequency-domain generation artefacts that evolve over time*. Our FTCA block is designed specifically for this.

3.5.2 Architecture

The FTCA block processes a clip $\mathcal{X} \in \mathbb{R}^{T \times H \times W \times 3}$ through two parallel branches whose outputs are fused via cross-attention.

RGB Branch. A pretrained ResNet3D-18 backbone (Kinetics-400) extracts spatio-temporal features:

$$\mathbf{F}_{\text{RGB}} = f_\theta(\mathcal{X}) \in \mathbb{R}^{T' \times d} \tag{13}$$

Frequency Branch. For each frame x_t, apply 2D DFT and log-compress the magnitude:

$$\mathbf{M}_t = \log(1 + |\mathcal{F}\{x_t^{\text{gray}}\}|) \tag{14}$$

A lightweight CNN encoder g_ϕ maps the sequence of frequency maps to temporal frequency embeddings:

$$\mathbf{F}_{\text{freq}} = g_\phi([\mathbf{M}_1, \ldots, \mathbf{M}_T]) \in \mathbb{R}^{T' \times d} \tag{15}$$

Cross-Attention Fusion. The RGB stream *queries* the frequency stream. This forces the model to learn: *"when I see this spatial motion pattern, what frequency artefact should I expect if this is a real camera?"*

$$\mathbf{Q} = \mathbf{F}_{\text{RGB}} W_Q, \quad \mathbf{K} = \mathbf{F}_{\text{freq}} W_K, \quad \mathbf{V} = \mathbf{F}_{\text{freq}} W_V \tag{16}$$

$$\mathbf{F}_{\text{fused}} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V} \tag{17}$$

$\mathbf{F}_{\text{fused}}$ is passed to a classification head producing the deepfake probability score $s_4 \in [0, 1]$.

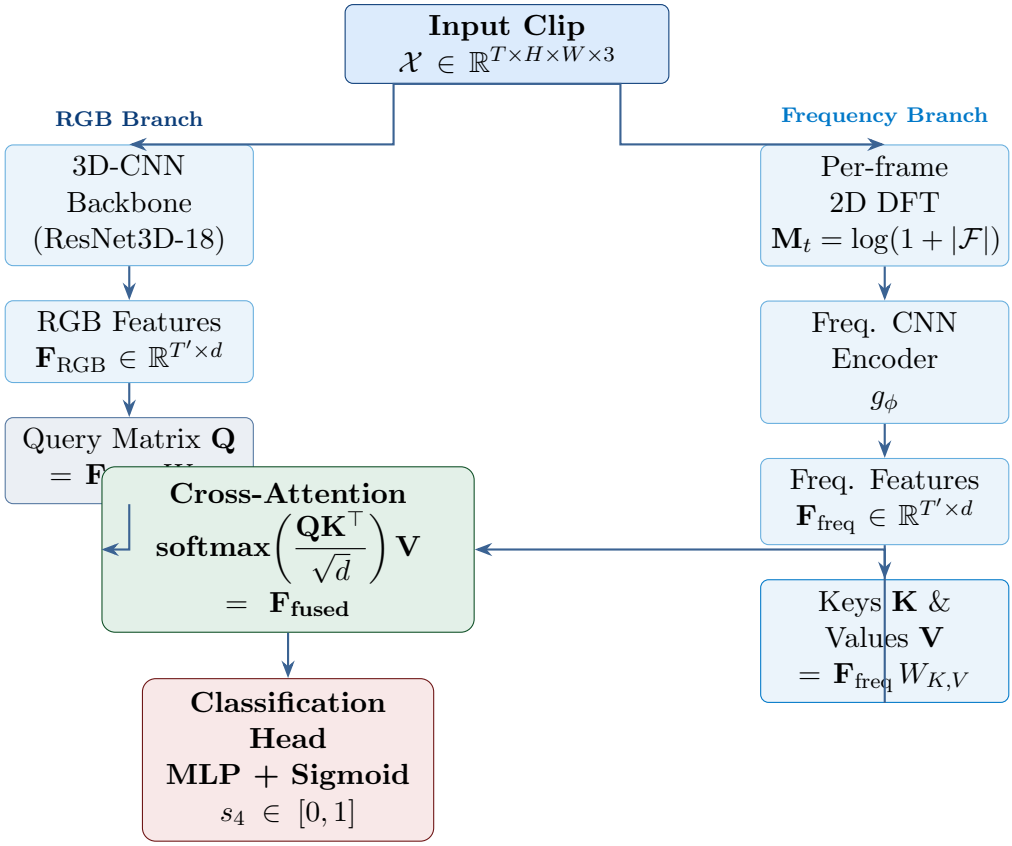

Figure 5: FTCA Block Architecture. The RGB branch captures spatio-temporal motion patterns; the Frequency branch captures DFT-domain generation artefacts. Cross-attention forces the model to correlate both signals temporally.

3.5.3 Training Objective

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{BCE}}(y, \hat{y})}_{\text{classification}} + \lambda_1 \underbrace{\mathcal{L}_{\text{freq-contrast}}(\mathbf{F}_{\text{freq}}, \mathbf{F}^{\text{real}}_{\text{freq}})}_{\text{pull real freq. embeddings together}} + \lambda_2 \underbrace{\mathcal{L}_{\text{temporal}}(\mathbf{F}_{\text{fused}}, \Delta t)}_{\text{penalise inconsistent freq. trajectory}} \tag{18}$$

4 Score Fusion and Final Decision

All active modules produce a fraud probability score $s_i \in [0, 1]$. The final risk score is:

$$r = \sigma\left(\sum_{i=0}^{4} w_i\, s_i + b\right) \tag{19}$$

Weights $\mathbf{w} = [w_0, w_1, w_2, w_3, w_4]$ and bias b are learned end-to-end during joint fine-tuning. A **hard-override rule** applies independently of r:

Hard Override Rule

If $s_3 > 0.95$ (rPPG biological gate fires with very high confidence), the session is **immediately and unconditionally rejected**, regardless of the final score r and regardless of what the FTCA engine outputs.

Table 3: Final Decision Thresholds

Risk Score r	Decision	Action
$r < 0.30$	Low risk	Proceed with KYC
$0.30 \leq r < 0.70$	Medium risk	Trigger secondary challenge
$r \geq 0.70$	High risk	Reject session; log for audit
$s_3 > 0.95$ (hard override)		Immediate reject

5 In-the-Wild Evaluation Dataset: NIT-KYC Wild (NKW-400)

5.1 Motivation and Contribution

Existing benchmark datasets (FaceForensics++, Celeb-DF, DFDC) are produced in controlled research environments and are well-known to the community. Models trained and evaluated exclusively on these datasets suffer from *dataset bias*: they may detect research-grade deepfakes but fail on content produced by real-world adversaries using newer or unknown generation pipelines.

To address this, we curate **NIT-KYC Wild (NKW-400)**: a novel evaluation dataset of 400 deepfake videos collected from public social media platforms (Instagram, YouTube) under real-world distribution, intended exclusively as a held-out test set.

5.2 Dataset Construction Protocol

1. **Collection.** Videos are sourced from publicly accessible Instagram Reels and YouTube Shorts tagged with known deepfake or synthetic media indicators.

2. **Human Perceptual Screening.** Each video is independently rated by $N = 5$ human evaluators asked: *"Can you tell that this face is AI-generated?"* Only videos where **all 5 evaluators answered No** (i.e., 100% human-indistinguishable) are retained. This is the defining property of NKW-400: it consists exclusively of **visually convincing, hard-negative samples** that fool human perception.

3. **Generation Method Annotation.** Each video is labelled with an inferred generation method (FaceSwap-class, Diffusion-based, Unknown-pipeline) based

on visual artefact signatures, resulting in a diversity of synthesis approaches.

4. **Quality and Diversity Filters.** Videos are filtered for: resolution $\geq$ 480p, duration $\geq$ 8 s, frontal face coverage $\geq$ 60% of frames, diverse ethnicity and lighting conditions.

5.3 Significance

Why NKW-400 Matters

- **Real-world distribution shift:** Videos were created by adversaries trying to be convincing, not researchers trying to build a dataset. This is harder than any existing benchmark.
- **Human-certified difficulty:** The perceptual screening ensures the evaluation is not trivially solvable by visual inspection.
- **Novel citable contribution:** NKW-400 is an independent dataset contribution — it can be referenced and released independently of the detection system.
- **Evaluation only:** NKW-400 is *not used in training*, eliminating data leakage concerns.

6 Codebase Architecture and Rationale

6.1 Directory Structure

```
AuthKYC/
|-- modules/
|    |-- telemetry.py        # Stage 0: FPS / timestamp analyser
|    |-- prnu.py             # Stage 1: BM3D denoiser +
   fingerprint
|    |-- replay.py           # Stage 2: 2D FFT Moire detector
|    |-- rppg.py             # Stage 3: CHROM extractor + SNR
   gate
|    |-- ftca/
|        |-- backbone.py     # ResNet3D-18 RGB branch
|        |-- freq_encoder.py# DFT map + CNN frequency branch
|        |-- cross_attn.py   # Cross-attention fusion module
|        |-- model.py        # Full FTCA model wrapper
|-- core_engine.py           # Orchestrator: routes frames to
   modules
|-- fusion.py                # Weighted score fusion layer
|-- main.py                  # FastAPI gateway (WebSocket
   endpoint)
|-- train/
|    |-- train_ftca.py       # FTCA training script
|    |-- train_fusion.py     # Joint fusion fine-tuning
|-- eval/
|    |-- benchmark.py        # Evaluation on all datasets + NKW
   -400
|    |-- ablation.py         # Per-module ablation runner
|-- docker/
|    |-- Dockerfile
|    |-- docker-compose.yml
```

6.2 Design Rationale

`modules/` — Strict Decoupling. Each module is a mathematically isolated class with a single interface: `score(frames) -> float`. This ensures that modifying the rPPG threshold cannot affect the FTCA training loop, and that individual modules can be unit-tested against known attack samples independently.

`core_engine.py` — The Orchestrator. Acts as the traffic cop. It extracts frames from the incoming stream, invokes modules in waterfall order, short-circuits on early rejection, and aggregates surviving scores into a JSON payload. It contains *no model logic*.

`main.py` — The FastAPI Gateway. A stateless REST and WebSocket server.

Receives a video upload or live stream, passes frames to `core_engine`, and returns a structured fraud-risk payload:

```
{
  "session_id": "uuid",
  "risk_score": 0.87,
  "decision": "REJECT",
  "module_scores": {
    "telemetry": 0.12,
    "prnu": 0.08,
    "replay": 0.05,
    "rppg": 0.91,    # <- triggered hard override
    "ftca": 0.83
  },
  "rejection_stage": "rPPG_hard_override",
  "latency_ms": 112
}
```

This structured output proves the system is **deployment-ready** for integration into any commercial fintech KYC environment via a standard REST API.

7 Evaluation Plan

7.1 Metrics

- **AUC-ROC**: Primary detection quality metric.
- **Equal Error Rate (EER)**: Target $< 3\%$ on FF++ (c23).
- **False Acceptance Rate (FAR)**: Target $< 1\%$ (banking requirement).
- **False Rejection Rate (FRR)**: Target $< 5\%$ (UX requirement).
- **End-to-end latency**: Target $< 150\,\text{ms}$.

7.2 Ablation Study

To demonstrate each module's independent contribution:

Table 4: Ablation Variants and Expected Insights

Variant	Modules Active
Baseline	Replay detector only (original system)
+Telemetry	Baseline + S0
+PRNU	Baseline + S0 + S1
+rPPG	Baseline + S0 + S1 + S3
+FTCA	Baseline + S0 + S1 + S3 + S4
Full system	**All modules + fusion**

Each variant is evaluated on all four attack categories and on NKW-400 to isolate per-module and cross-module gains.

8 Summary of Novel Contributions

Table 5: Contribution Map

Contribution	Description	Significance
rPPG Liveness Gate	Cardiovascular pulse extraction used as a hard biological liveness filter in a KYC pipeline	Patent anchor
FTCA Architecture	Custom 3D-CNN + Frequency-branch cross-attention block for temporal deepfake detection	Publication anchor
NKW-400 Dataset	400 human-certified, visually indistinguishable in-the-wild deepfake videos for adversarial benchmarking	Dataset contribution
Waterfall PAD Pipeline	Layered Presentation Attack Detection gateway with progressive complexity and early rejection	Systems contribution

References

[1] A. Rössler *et al.*, "FaceForensics++: Learning to Detect Manipulated Facial Images," *Proc. ICCV*, 2019.

[2] G. de Haan and V. Jeanne, "Robust Pulse Rate From Chrominance-Based rPPG," *IEEE Trans. Biomed. Eng.*, vol. 60, no. 10, 2013.

[3] J. Lukáš, J. Fridrich, and M. Goljan, "Digital Camera Identification From Sensor Pattern Noise," *IEEE Trans. Inf. Forensics Secur.*, vol. 1, no. 2, 2006.

[4] Y. Li *et al.*, "Celeb-DF: A Large-Scale Challenging Dataset for DeepFake Forensics," *Proc. CVPR*, 2020.

[5] H. Liu *et al.*, "Detecting Deepfake Videos with Temporal Dropout 3DCNN," *Proc. IJCAI*, 2021.

[6] ISO/IEC 30107-3:2023, *Information technology — Biometric presentation attack detection — Part 3: Testing and reporting.*